

DeepGEMM

Deep Dive into DeepSeek — Chapter 23

Yin Yang

Foundation Books Project

The one rule of this chapter

The equation $D = C + AB^\top$ never changes.

Everything else about a GEMM call is a choice DeepGEMM must resolve.

- Attention projections, dense MLPs, MoE experts, and gradients all reduce to batches of matrix products.
- Payload dtype, scale metadata, operand layout, architecture path, grouping, and JIT state all change *how* the product is computed — never *which* product is declared.
- A timing number means nothing until a fixed configuration passes its declared numerical tolerance.

The one rule of this chapter

The equation $D = C + AB^\top$ never changes.

Everything else about a GEMM call is a choice DeepGEMM must resolve.

- Attention projections, dense MLPs, MoE experts, and gradients all reduce to batches of matrix products.
- Payload dtype, scale metadata, operand layout, architecture path, grouping, and JIT state all change *how* the product is computed — never *which* product is declared.
- A timing number means nothing until a fixed configuration passes its declared numerical tolerance.

Goal: follow one 6×8 by 12×8 running example through BF16, low-precision, grouped, and JIT-selected paths, and see where each piece lives in the released DeepGEMM source.

Roadmap

The GEMM contract and its BF16 reference

Low-precision GEMM, scale layout, and the call interface

Runtime JIT and cold/warm measurement

One generated specialization, grouped experts, and adjacent kernels

Evidence discipline and what comes next

The GEMM contract and its BF16 reference

One equation, many call sites

Definition (GEMM)

$A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{N \times K}$, $C, D \in \mathbb{R}^{M \times N}$, and $D = C + AB^\top$. With no C , read $C = 0$.

Running example: $M = 6, K = 8, N = 12$. Output has $MN = 72$ coordinates, each a length-8 dot product: $2MNK = 1152$ scalar floating-point operations.

One equation, many call sites

Definition (GEMM)

$A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{N \times K}$, $C, D \in \mathbb{R}^{M \times N}$, and $D = C + AB^\top$. With no C , read $C = 0$.

Running example: $M = 6, K = 8, N = 12$. Output has $MN = 72$ coordinates, each a length-8 dot product: $2MNK = 1152$ scalar floating-point operations.

Attention projections, dense MLPs, MoE experts, and gradients share this shape rule. They diverge in dtype, layout, grouping, and JIT state — not in the declared product.

A reference before anything gets low-precision

$$A = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \implies D^{\text{ref}} = \begin{bmatrix} 6 & 10 \\ -1 & -2 \end{bmatrix}.$$

- BF16 inputs, BF16 or FP32 output, and a supplied C that matches the output dtype and layout.
- A DeepGEMM kernel result D^{ker} is compared against this reference under a declared metric — not judged by intuition.

A reference before anything gets low-precision

$$A = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \implies D^{\text{ref}} = \begin{bmatrix} 6 & 10 \\ -1 & -2 \end{bmatrix}.$$

- BF16 inputs, BF16 or FP32 output, and a supplied C that matches the output dtype and layout.
- A DeepGEMM kernel result D^{ker} is compared against this reference under a declared metric — not judged by intuition.

FP8 and FP4 paths reuse this same logical product. They change what is stored and moved, not the equation being checked.

Low-precision GEMM, scale layout, and the call interface

Payload plus scale metadata, not a dtype label

Path	Textbook object	Caution
BF16	stored approximate value	still not exact real arithmetic

Payload plus scale metadata, not a dtype label

Path	Textbook object	Caution
BF16	stored approximate value	still not exact real arithmetic
FP8	payload + scale metadata	cite range/scaling claims

Payload plus scale metadata, not a dtype label

Path	Textbook object	Caution
BF16	stored approximate value	still not exact real arithmetic
FP8	payload + scale metadata	cite range/scaling claims
FP4/NVFP4	smaller payload + scale metadata	cite hardware-format claims

Payload plus scale metadata, not a dtype label

Path	Textbook object	Caution
BF16	stored approximate value	still not exact real arithmetic
FP8	payload + scale metadata	cite range/scaling claims
FP4/NVFP4	smaller payload + scale metadata	cite hardware-format claims

UE8M0: unsigned, 8 exponent bits, 0 mantissa bits. Stored byte c decodes to $s(c) = 2^{c-127}$: only positive powers of two, no zero, no infinity.

Payload plus scale metadata, not a dtype label

Path	Textbook object	Caution
BF16	stored approximate value	still not exact real arithmetic
FP8	payload + scale metadata	cite range/scaling claims
FP4/NVFP4	smaller payload + scale metadata	cite hardware-format claims

UE8M0: unsigned, 8 exponent bits, 0 mantissa bits. Stored byte c decodes to $s(c) = 2^{c-127}$: only positive powers of two, no zero, no infinity.

A represented value is $\hat{x}_{i,j} = s_{i,b(j)} q_{i,j}$. Payload and scale block must agree before the number means anything.

Layout: same logical scales, a different memory object

Definition (Layout)

A layout maps logical tensor indices to memory locations and strides — separate from shape and represented value.

- SM90 scale factors: FP32, TMA-aligned and transposed.

Layout: same logical scales, a different memory object

Definition (Layout)

A layout maps logical tensor indices to memory locations and strides — separate from shape and represented value.

- SM90 scale factors: FP32, TMA-aligned and transposed.
- SM100 scale factors: four packed UE8M0 codes per 32-bit word.

Layout: same logical scales, a different memory object

Definition (Layout)

A layout maps logical tensor indices to memory locations and strides — separate from shape and represented value.

- SM90 scale factors: FP32, TMA-aligned and transposed.
- SM100 scale factors: four packed UE8M0 codes per 32-bit word.
- Example: $[\frac{1}{2}, 1, 2, 4] \rightarrow [7e, 7f, 80, 81] \rightarrow 0x81807f7e$. Reversing the byte order is a layout error even though every code stays legal.

Layout: same logical scales, a different memory object

Definition (Layout)

A layout maps logical tensor indices to memory locations and strides — separate from shape and represented value.

- SM90 scale factors: FP32, TMA-aligned and transposed.
- SM100 scale factors: four packed UE8M0 codes per 32-bit word.
- Example: $[\frac{1}{2}, 1, 2, 4] \rightarrow [7\text{e}, 7\text{f}, 80, 81] \rightarrow 0\text{x}81807\text{f}7\text{e}$. Reversing the byte order is a layout error even though every code stays legal.

Padding and packing are legal only when they never add real rows or silently change which scale a payload block reconstructs with.

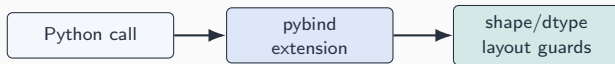
Python call, checked guards, JIT-selected kernel

Python call

Python call, checked guards, JIT-selected kernel



Python call, checked guards, JIT-selected kernel



Python call, checked guards, JIT-selected kernel



Python call, checked guards, JIT-selected kernel



- `sm90` and `sm100` name GPU architecture targets, not SM counts. `SM90` accumulates WGMMMA into registers; `SM100` accumulates `tcgen05.mma` into Tensor Memory (TMEM).

Python call, checked guards, JIT-selected kernel



- `sm90` and `sm100` name GPU architecture targets, not SM counts. `SM90` accumulates WGMMMA into registers; `SM100` accumulates `tcgen05.mma` into Tensor Memory (TMEM).
- Both paths reunite at the same logical D ; neither architecture name changes the oracle.

Python call, checked guards, JIT-selected kernel



- `sm90` and `sm100` name GPU architecture targets, not SM counts. `SM90` accumulates `WGMMA` into registers; `SM100` accumulates `tcgen05.mma` into Tensor Memory (TMEM).
- Both paths reunite at the same logical D ; neither architecture name changes the oracle.

The N/T suffix (`NT`, `NN`, `TN`, `TT`) names how each passed operand enters the product — a view can change shape/strides without moving bytes, but the caller still owns any *material* transpose.

Runtime JIT and cold/warm measurement

Cold call, warm call: two different events

Definition (JIT specialization)

Runtime compilation or loading of a kernel selected for one combination of operation, shape, dtype, layout, architecture, and compiler settings.

Cold call, warm call: two different events

Definition (JIT specialization)

Runtime compilation or loading of a kernel selected for one combination of operation, shape, dtype, layout, architecture, and compiler settings.

- Selection is deterministic: enumerate legal plans, reject illegal ones, rank survivors with a source-defined cost model — not a hidden timing search on the request path.

Cold call, warm call: two different events

Definition (JIT specialization)

Runtime compilation or loading of a kernel selected for one combination of operation, shape, dtype, layout, architecture, and compiler settings.

- Selection is deterministic: enumerate legal plans, reject illegal ones, rank survivors with a source-defined cost model — not a hidden timing search on the request path.
- A cache miss compiles into a unique temp directory, then attempts an atomic rename into the shared digest cache; the loser discards its copy.

Cold call, warm call: two different events

Definition (JIT specialization)

Runtime compilation or loading of a kernel selected for one combination of operation, shape, dtype, layout, architecture, and compiler settings.

- Selection is deterministic: enumerate legal plans, reject illegal ones, rank survivors with a source-defined cost model — not a hidden timing search on the request path.
- A cache miss compiles into a unique temp directory, then attempts an atomic rename into the shared digest cache; the loser discards its copy.

A benchmark that mixes first-call compilation with warm steady-state execution is not measuring one well-defined event. [OBSERVED: Panther H200 precision profiler packet]

From concept to code: the JIT cache key

```
from DeepGEMM/csrc/jit/compiler.hpp

if (const auto env_cache_dir_path =
    get_env<std::string>("DG_JIT_CACHE_DIR"); not env_cache_dir_path.empty())
    cache_dir_path = env_cache_dir_path;

// NVRTCCompiler: requires NVRTC >= 12.3
DG_HOST_ASSERT((major > 12 or (major == 12 and minor >= 3)) and ...);
```


From concept to code: the JIT cache key

```
from DeepGEMM/csrc/jit/compiler.hpp

if (const auto env_cache_dir_path =
    get_env<std::string>("DG_JIT_CACHE_DIR"); not env_cache_dir_path.empty())
    cache_dir_path = env_cache_dir_path;

// NVRTCCompiler: requires NVRTC >= 12.3
DG_HOST_ASSERT((major > 12 or (major == 12 and minor >= 3)) and ...);
```

DG_JIT_CACHE_DIR makes the digest cache location explicit; the NVRTC branch is one of two compiler paths that both end at a cubin. [INSPECTED: DeepGEMM csrc/jit/compiler.hpp]

**One generated specialization,
grouped experts, and adjacent
kernels**

A retained SM90 plan, not a generic launch

$N = 2112$, $K = 7168$, M dynamic; $128 \times 192 \times 128$ CTA tile, four stages, two-CTA multicast on B , 132 persistent slots over $32 \times 11 = 352$ output tiles.

A retained SM90 plan, not a generic launch

$N = 2112$, $K = 7168$, M dynamic; $128 \times 192 \times 128$ CTA tile, four stages, two-CTA multicast on B , 132 persistent slots over $32 \times 11 = 352$ output tiles.

- Persistent slot p claims logical indices $p, p + S, p + 2S, \dots$; a swizzle then maps each claimed index to a tile coordinate.
- Tile width 192 crosses the SM90 128×128 weight-scale boundary: 64 columns use one B scale, 128 use the next.

A retained SM90 plan, not a generic launch

$N = 2112$, $K = 7168$, M dynamic; $128 \times 192 \times 128$ CTA tile, four stages, two-CTA multicast on B , 132 persistent slots over $32 \times 11 = 352$ output tiles.

- Persistent slot p claims logical indices $p, p + S, p + 2S, \dots$; a swizzle then maps each claimed index to a tile coordinate.
- Tile width 192 crosses the SM90 128×128 weight-scale boundary: 64 columns use one B scale, 128 use the next.

$\text{calc_diff} = 6.84 \times 10^{-4} < 10^{-3}$; cold 16.879s vs. warm mean 0.12456ms. [OBSERVED: Panther H200 precision profiler packet]

Grouped GEMM: routed rows, not a new product

Six tokens, top-2 routing, three experts: $(M_0, M_1, M_2) = (5, 4, 3)$.

$$D_0 = A_0 B_0^\top$$

Grouped GEMM: routed rows, not a new product

Six tokens, top-2 routing, three experts: $(M_0, M_1, M_2) = (5, 4, 3)$.

$$D_0 = A_0 B_0^\top \quad D_1 = A_1 B_1^\top \quad D_2 = A_2 B_2^\top.$$

Grouped GEMM: routed rows, not a new product

Six tokens, top-2 routing, three experts: $(M_0, M_1, M_2) = (5, 4, 3)$.

$$D_0 = A_0 B_0^\top \quad D_1 = A_1 B_1^\top \quad D_2 = A_2 B_2^\top.$$

- Contiguous layout: concatenated valid rows plus segment boundaries.

Grouped GEMM: routed rows, not a new product

Six tokens, top-2 routing, three experts: $(M_0, M_1, M_2) = (5, 4, 3)$.

$$D_0 = A_0 B_0^\top \quad D_1 = A_1 B_1^\top \quad D_2 = A_2 B_2^\top.$$

- Contiguous layout: concatenated valid rows plus segment boundaries.
- Masked layout: fixed per-expert capacity plus a valid-row count.
- K-grouped GEMM varies the *reduction* extent K_g instead, holding M, N fixed — a different axis, a different use case.

Grouped GEMM: routed rows, not a new product

Six tokens, top-2 routing, three experts: $(M_0, M_1, M_2) = (5, 4, 3)$.

$$D_0 = A_0 B_0^\top \quad D_1 = A_1 B_1^\top \quad D_2 = A_2 B_2^\top.$$

- Contiguous layout: concatenated valid rows plus segment boundaries.
- Masked layout: fixed per-expert capacity plus a valid-row count.
- K-grouped GEMM varies the *reduction* extent K_g instead, holding M, N fixed — a different axis, a different use case.

The checked M-grouped wrappers have no additive C_e : they compute $D_e = A_e B_e^\top$, not the dense chapter's $D = C + AB^\top$.

Adjacent surfaces keep their own oracle

Surface	Output relation
Einsum (bmk, bnk → mn)	reduces batch b ; one m, n matrix, every split must reach the shared FP32 destination

Adjacent surfaces keep their own oracle

Surface	Output relation
Einsum ($\text{bnk}, \text{bnk} \rightarrow \text{mn}$)	reduces batch b ; one m, n matrix, every split must reach the shared FP32 destination
HC prenorm	one activation read yields projection D and row square-sum s ; both must use the same complete reduction

Adjacent surfaces keep their own oracle

Surface	Output relation
Einsum ($\text{bnk}, \text{bnk} \rightarrow \text{mn}$)	reduces batch b ; one m, n matrix, every split must reach the shared FP32 destination
HC prenorm	one activation read yields projection D and row square-sum s ; both must use the same complete reduction
MQA/indexer logits	weighted-ReLU score over candidate cache positions, not an $M \times N$ GEMM output

Adjacent surfaces keep their own oracle

Surface	Output relation
Einsum (bmk, bnk→mn)	reduces batch b ; one m, n matrix, every split must reach the shared FP32 destination
HC prenorm	one activation read yields projection D and row square-sum s ; both must use the same complete reduction
MQA/indexer logits	weighted-ReLU score over candidate cache positions, not an $M \times N$ GEMM output

Each adjacent kernel resembles GEMM in machinery — FP8 payloads, scale metadata, JIT — but owes a different declared result. Dense-GEMM tolerances do not validate any of them.

Evidence discipline and what comes next

What this chapter's evidence does and does not cover

Covers

- Reported interfaces and kernel families.
- Inspected JIT and compiled-artifact surfaces.
- Derived dense and grouped-work oracles.
- Observed H200 timing and launch attribution, one fixed configuration at a time.

What this chapter's evidence does and does not cover

Covers

- Reported interfaces and kernel families.
- Inspected JIT and compiled-artifact surfaces.
- Derived dense and grouped-work oracles.
- Observed H200 timing and launch attribution, one fixed configuration at a time.

Does not cover

- A cross-architecture speed ranking.
- Model-backed V3.2 indexer serving (671B artifact exceeds four H200s).
- Einsum and HC prenorm timing — released and tested, but unobserved here.

What this chapter's evidence does and does not cover

Covers

- Reported interfaces and kernel families.
- Inspected JIT and compiled-artifact surfaces.
- Derived dense and grouped-work oracles.
- Observed H200 timing and launch attribution, one fixed configuration at a time.

Does not cover

- A cross-architecture speed ranking.
- Model-backed V3.2 indexer serving (671B artifact exceeds four H200s).
- Einsum and HC prenorm timing — released and tested, but unobserved here.

A comparable timing identity holds the operator, shape, dtype, scale recipe, layout, architecture, and timing protocol fixed. Change one field and the rows describe different configurations.

Concept-to-code map for the chapter

- dense/grouped call surface → `DeepGEMM/csrc/apis/gemm.hpp`

Concept-to-code map for the chapter

- **dense/grouped call surface** → `DeepGEMM/csrc/apis/gemm.hpp`
- **extension boundary** → `DeepGEMM/csrc/python_api.cpp`

Concept-to-code map for the chapter

- **dense/grouped call surface** → `DeepGEMM/csrc/apis/gemm.hpp`
- **extension boundary** → `DeepGEMM/csrc/python_api.cpp`
- **JIT compiler and cache digest** → `DeepGEMM/csrc/jit/compiler.hpp`

Concept-to-code map for the chapter

- **dense/grouped call surface** → `DeepGEMM/csrc/apis/gemm.hpp`
- **extension boundary** → `DeepGEMM/csrc/python_api.cpp`
- **JIT compiler and cache digest** → `DeepGEMM/csrc/jit/compiler.hpp`
- **deterministic SM90 plan selection** → `DeepGEMM/csrc/jit_kernels/heuristics/sm90.hpp`
- **block-scale and tolerance oracle** → `code/chapter_deepgemm/gemm_quant_reference.py`

Concept-to-code map for the chapter

- **dense/grouped call surface** → `DeepGEMM/csrc/apis/gemm.hpp`
- **extension boundary** → `DeepGEMM/csrc/python_api.cpp`
- **JIT compiler and cache digest** → `DeepGEMM/csrc/jit/compiler.hpp`
- **deterministic SM90 plan selection** → `DeepGEMM/csrc/jit_kernels/heuristics/sm90.hpp`
- **block-scale and tolerance oracle** → `code/chapter_deepgemm/gemm_quant_reference.py`

Fix the equation. Then resolve representation, layout, grouping, and JIT state — and check every claim against a declared oracle.

Next: the Mega MoE chapter widens the grouped-expert boundary to multi-rank dispatch, two expert linears, activation, and combine.